

JAVA LAB RECORD

TASK – 05

Java Collection Framework: Important Methods by Interface

1. BASIC INFORMATION

Experiment No.	05
Date	06/10/2026
Title	Java Collection Framework: Important Methods by Interface

2. OBJECTIVE / AIM

To study and implement important methods of the Java Collection Framework using Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator.

3. SYSTEM REQUIREMENTS

Hardware Requirements: Standard PC/Laptop; Intel Core i3/i5 or equivalent processor; minimum 4 GB RAM; minimum 500 MB free storage.

Software Requirements: Windows/Linux/macOS; JDK 17 or JDK 21; Eclipse / IntelliJ IDEA / NetBeans / VS Code; Command Prompt / Terminal.

4. PROBLEM STATEMENT

Develop and implement Java programs to demonstrate important methods of the Java Collection Framework. The programs demonstrate Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator operations.

5. ALGORITHM / LOGIC DESCRIPTION

1. Start the program.
2. Import the java.util.* package.
3. Create the required collection interface reference.
4. Create an appropriate implementation object.
5. Add the required elements.
6. Apply the methods associated with the interface.
7. Display the collection before and after modification.
8. Perform searching operations wherever required.
9. Perform insertion and deletion operations wherever required.
10. Perform sorting or ordering operations wherever applicable.
11. Use Iterator or ListIterator for traversal.

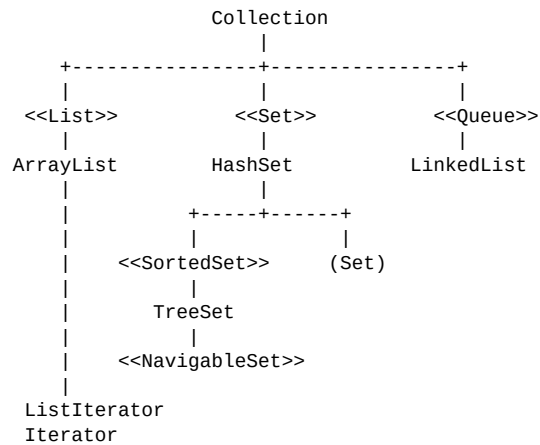
12. Display the final result.

13. Stop the program.

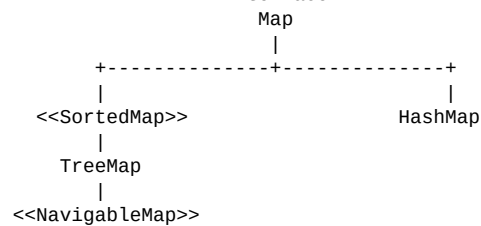
6. UML / CLASS DIAGRAM

The following UML-style hierarchy represents the main interfaces and implementation classes used in the 12 programs.

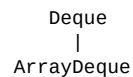
<<interface>>



<<interface>>



<<interface>>



Note: Map is a separate hierarchy from Collection. HashMap, SortedMap, NavigableMap, and TreeMap are therefore shown separately.

PROGRAM 1 – COLLECTION METHODS

File Name: CollectionMethods.java

Source Code

```
package packagecode.java;
import java.util.*;

public class CollectionMethods {
    public static void main(String[] args) {
        Collection<String> c1 = new ArrayList<>();
        c1.add("Apple");
        c1.add("Banana");
        c1.add("Mango");
        System.out.println("Collection: " + c1);

        Collection<String> c2 = new ArrayList<>();
        c2.add("Orange");
        c2.add("Grapes");
        c1.addAll(c2);
        System.out.println("After addAll(): " + c1);
        System.out.println("Contains Mango: " + c1.contains("Mango"));
        System.out.println("Contains all c2: " + c1.containsAll(c2));
        System.out.println("Size: " + c1.size());
        System.out.println("Is Empty: " + c1.isEmpty());
        c1.remove("Banana");
        System.out.println("After remove(): " + c1);
        c1.removeAll(c2);
        System.out.println("After removeAll(): " + c1);

        System.out.println("Using Iterator:");
        Iterator<String> itr = c1.iterator();
        while (itr.hasNext()) {
            System.out.println(itr.next());
        }

        c1.clear();
        System.out.println("After clear(): " + c1);
    }
}
```

Output

```
Collection: [Apple, Banana, Mango]
After addAll(): [Apple, Banana, Mango, Orange, Grapes]
Contains Mango: true
Contains all c2: true
Size: 5
Is Empty: false
After remove(): [Apple, Mango, Orange, Grapes]
After removeAll(): [Apple, Mango]
Using Iterator:
Apple
Mango
After clear(): []
```

PROGRAM 2 – DEQUE METHODS

File Name: DequeMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class DequeMethod {
    public static void main(String[] args) {
        Deque<String> deque = new ArrayDeque<>();
        deque.addFirst("B");
        deque.addLast("C");
        deque.addFirst("A");
        deque.addLast("D");
        System.out.println("Deque: " + deque);

        deque.offerFirst("Start");
        deque.offerLast("End");
        System.out.println("After offer: " + deque);

        System.out.println("First: " + deque.peekFirst());
        System.out.println("Last: " + deque.peekLast());

        System.out.println("Removed First: " + deque.removeFirst());
        System.out.println("Removed Last: " + deque.removeLast());

        System.out.println("Poll First: " + deque.pollFirst());
        System.out.println("Poll Last: " + deque.pollLast());

        System.out.println("Final Deque: " + deque);
    }
}
```

Output

```
Deque: [A, B, C, D]
After offer: [Start, A, B, C, D, End]
First: Start
Last: End
Removed First: Start
Removed Last: End
Poll First: A
Poll Last: D
Final Deque: [B, C]
```

PROGRAM 3 – ITERATOR METHODS

File Name: IteratorMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class IteratorMethod {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Mango");
        list.add("Orange");

        Iterator<String> itr = list.iterator();
        System.out.println("Elements:");
        while (itr.hasNext()) {
            String item = itr.next();
            System.out.println(item);
            if (item.equals("Banana")) {
                itr.remove();
            }
        }

        System.out.println("After Iterator remove(): " + list);

        System.out.println("Using forEachRemaining():");
        Iterator<String> itr2 = list.iterator();
        itr2.forEachRemaining(System.out::println);
    }
}
```

Output

```
Elements:
Apple
Banana
Mango
Orange
After Iterator remove(): [Apple, Mango, Orange]
Using forEachRemaining():
Apple
Mango
Orange
```

PROGRAM 4 – LISTITERATOR METHODS

File Name: ListIteratorMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class ListIteratorMethod {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Mango");

        ListIterator<String> itr = list.listIterator();

        System.out.println("Forward Traversal:");
        while (itr.hasNext()) {
            System.out.println(
                "Index " + itr.nextIndex() + ": " + itr.next()
            );
        }

        System.out.println("\nBackward Traversal:");
        while (itr.hasPrevious()) {
            System.out.println(
                "Index " + itr.previousIndex() + ": " + itr.previous()
            );
        }

        itr = list.listIterator();
        itr.next();
        itr.set("Orange");
        System.out.println("\nAfter set(): " + list);

        itr.add("Grapes");
        System.out.println("After add(): " + list);

        itr.previous();
        itr.remove();
        System.out.println("After remove(): " + list);
    }
}
```

Output

Forward Traversal:
Index 0: Apple
Index 1: Banana
Index 2: Mango

Backward Traversal:
Index 2: Mango
Index 1: Banana
Index 0: Apple

After set(): [Orange, Banana, Mango]
After add(): [Orange, Grapes, Banana, Mango]
After remove(): [Orange, Banana, Mango]

PROGRAM 5 – LIST METHODS

File Name: ListMehods.java

Source Code

```
package packagecode.java;
import java.util.*;

public class ListMehods {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("Apple");
        list.add("Banana");
        list.add("Mango");
        list.add("Banana");
        System.out.println("List: " + list);

        list.add(1, "Orange");
        System.out.println("After add(1, Orange): " + list);

        System.out.println("Element at index 2: " + list.get(2));

        list.set(2, "Grapes");
        System.out.println("After set(): " + list);

        list.remove(1);
        System.out.println("After remove(1): " + list);

        System.out.println("First index of Banana: " + list.indexOf("Banana"));
        System.out.println("Last index of Banana: " + list.lastIndexOf("Banana"));
        System.out.println("SubList: " + list.subList(0, 2));

        list.sort(Comparator.naturalOrder());
        System.out.println("After sorting: " + list);
    }
}
```

Output

```
List: [Apple, Banana, Mango, Banana]
After add(1, Orange): [Apple, Orange, Banana, Mango, Banana]
Element at index 2: Banana
After set(): [Apple, Orange, Grapes, Mango, Banana]
After remove(1): [Apple, Grapes, Mango, Banana]
First index of Banana: 3
Last index of Banana: 3
SubList: [Apple, Grapes]
After sorting: [Apple, Banana, Grapes, Mango]
```

PROGRAM 6 – MAP METHODS

File Name: MapMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class MapMethod {
    public static void main(String[] args) {
        Map<Integer, String> map = new HashMap<>();
        map.put(101, "Apple");
        map.put(102, "Banana");
        map.put(103, "Mango");
        System.out.println("Map: " + map);

        System.out.println("Value of key 102: " + map.get(102));
        System.out.println("Contains key 101: " + map.containsKey(101));
        System.out.println("Contains value Mango: " + map.containsValue("Mango"));
        System.out.println("Size: " + map.size());
        System.out.println("Keys: " + map.keySet());
        System.out.println("Values: " + map.values());
        System.out.println("Entries: " + map.entrySet());

        map.remove(102);
        System.out.println("After remove(): " + map);
        System.out.println("Is Empty: " + map.isEmpty());

        map.clear();
        System.out.println("After clear(): " + map);
    }
}
```

Output

```
Map: {101=Apple, 102=Banana, 103=Mango}
Value of key 102: Banana
Contains key 101: true
Contains value Mango: true
Size: 3
Keys: [101, 102, 103]
Values: [Apple, Banana, Mango]
Entries: [101=Apple, 102=Banana, 103=Mango]
After remove(): {101=Apple, 103=Mango}
Is Empty: false
After clear(): {}
```

Note: HashMap/HashSet do not guarantee a fixed iteration order; the displayed element order may vary between Java implementations/runs.

PROGRAM 7 – NAVIGABLEMAP METHODS

File Name: NavigableMapMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class NavigableMapMethod {
    public static void main(String[] args) {
        NavigableMap<Integer, String> map = new TreeMap<>();
        map.put(10, "Apple");
        map.put(20, "Banana");
        map.put(30, "Mango");
        map.put(40, "Orange");
        map.put(50, "Grapes");

        System.out.println("NavigableMap: " + map);
        System.out.println("Lower Key than 30: " + map.lowerKey(30));
        System.out.println("Floor Key of 30: " + map.floorKey(30));
        System.out.println("Ceiling Key of 35: " + map.ceilingKey(35));
        System.out.println("Higher Key than 30: " + map.higherKey(30));
        System.out.println("First Entry: " + map.firstEntry());
        System.out.println("Last Entry: " + map.lastEntry());
        System.out.println("Poll First Entry: " + map.pollFirstEntry());
        System.out.println("Poll Last Entry: " + map.pollLastEntry());
        System.out.println("Descending Map: " + map.descendingMap());
    }
}
```

Output

```
NavigableMap: {10=Apple, 20=Banana, 30=Mango, 40=Orange, 50=Grapes}
Lower Key than 30: 20
Floor Key of 30: 30
Ceiling Key of 35: 40
Higher Key than 30: 40
First Entry: 10=Apple
Last Entry: 50=Grapes
Poll First Entry: 10=Apple
Poll Last Entry: 50=Grapes
Descending Map: {40=Orange, 30=Mango, 20=Banana}
```

PROGRAM 8 – NAVIGABLESET METHODS

File Name: NavigableSetMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class NavigableSetMethod {
    public static void main(String[] args) {
        NavigableSet<Integer> set = new TreeSet<>();
        set.add(10);
        set.add(20);
        set.add(30);
        set.add(40);
        set.add(50);

        System.out.println("NavigableSet: " + set);
        System.out.println("Lower than 30: " + set.lower(30));
        System.out.println("Floor of 30: " + set.floor(30));
        System.out.println("Ceiling of 35: " + set.ceiling(35));
        System.out.println("Higher than 30: " + set.higher(30));
        System.out.println("Poll First: " + set.pollFirst());
        System.out.println("Poll Last: " + set.pollLast());
        System.out.println("After polling: " + set);
        System.out.println("Descending Set: " + set.descendingSet());
    }
}
```

Output

```
NavigableSet: [10, 20, 30, 40, 50]
Lower than 30: 20
Floor of 30: 30
Ceiling of 35: 40
Higher than 30: 40
Poll First: 10
Poll Last: 50
After polling: [20, 30, 40]
Descending Set: [40, 30, 20]
```

PROGRAM 9 – QUEUE METHODS

File Name: QueueMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class QueueMethod{
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();
        queue.add("A");
        queue.add("B");
        queue.add("C");
        System.out.println("Queue: " + queue);

        queue.offer("D");
        System.out.println("After offer(): " + queue);

        System.out.println("Head using element(): " + queue.element());
        System.out.println("Head using peek(): " + queue.peek());

        System.out.println("Removed using remove(): " + queue.remove());
        System.out.println("Removed using poll(): " + queue.poll());

        System.out.println("Queue after removal: " + queue);
    }
}
```

Output

```
Queue: [A, B, C]
After offer(): [A, B, C, D]
Head using element(): A
Head using peek(): A
Removed using remove(): A
Removed using poll(): B
Queue after removal: [C, D]
```

PROGRAM 10 – SET METHODS

File Name: SetMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class SetMethod {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>();
        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Apple");
        System.out.println("Set: " + set);

        System.out.println("Contains Banana: " + set.contains("Banana"));
        System.out.println("Size: " + set.size());
        System.out.println("Is Empty: " + set.isEmpty());

        set.remove("Mango");
        System.out.println("After remove(): " + set);

        set.clear();
        System.out.println("After clear(): " + set);
    }
}
```

Output

```
Set: [Apple, Banana, Mango]
Contains Banana: true
Size: 3
Is Empty: false
After remove(): [Apple, Banana]
After clear(): []
```

Note: HashMap/HashSet do not guarantee a fixed iteration order; the displayed element order may vary between Java implementations/runs.

PROGRAM 11 – SORTEDMAP METHODS

File Name: SortedMapMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class SortedMapMethod {
    public static void main(String[] args) {
        SortedMap<Integer, String> map = new TreeMap<>();
        map.put(30, "Mango");
        map.put(10, "Apple");
        map.put(40, "Orange");
        map.put(20, "Banana");

        System.out.println("SortedMap: " + map);
        System.out.println("First Key: " + map.firstKey());
        System.out.println("Last Key: " + map.lastKey());
        System.out.println("HeadMap(<30): " + map.headMap(30));
        System.out.println("TailMap(20): " + map.tailMap(20));
        System.out.println("SubMap(10, 40): " + map.subMap(10, 40));
        System.out.println("Comparator: " + map.comparator());
    }
}
```

Output

```
SortedMap: {10=Apple, 20=Banana, 30=Mango, 40=Orange}
First Key: 10
Last Key: 40
HeadMap(<30): {10=Apple, 20=Banana}
TailMap(20): {20=Banana, 30=Mango, 40=Orange}
SubMap(10, 40): {10=Apple, 20=Banana, 30=Mango}
Comparator: null
```

PROGRAM 12 – SORTEDSET METHODS

File Name: SortedSetMethod.java

Source Code

```
package packagecode.java;
import java.util.*;

public class SortedSetMethod{
    public static void main(String[] args) {
        SortedSet<Integer> set = new TreeSet<>();
        set.add(50);
        set.add(20);
        set.add(40);
        set.add(10);
        set.add(30);

        System.out.println("SortedSet: " + set);
        System.out.println("First element: " + set.first());
        System.out.println("Last element: " + set.last());
        System.out.println("HeadSet(<30): " + set.headSet(30));
        System.out.println("TailSet(30): " + set.tailSet(30));
        System.out.println("SubSet(20, 50): " + set.subSet(20, 50));
        System.out.println("Comparator: " + set.comparator());
    }
}
```

Output

```
SortedSet: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
HeadSet(<30): [10, 20]
TailSet(30): [30, 40, 50]
SubSet(20, 50): [20, 30, 40]
Comparator: null
```

8. COMPILATION AND EXECUTION

All programs contain the package declaration `package packagecode.java;`. Therefore, place the classes inside the corresponding `packagecode.java` package in Eclipse.

Eclipse: Create/open the Java project → create package `packagecode.java` → create each class → paste the corresponding source code → Save → Run As → Java Application.

Command Line:

```
javac packagecode/java/CollectionMethods.java
java packagecode.java.CollectionMethods
```

9. REAL-TIME APPLICATIONS

Collection: Storing and processing groups of objects such as students, products, and employees.

List: Ordered data with duplicate values, such as shopping-cart items and playlists.

Set: Unique values such as usernames, email addresses, and IDs.

SortedSet: Unique elements maintained in sorted order.

NavigableSet: Finding elements lower than, equal to, or higher than a specified value.

Queue: Printer queues, task scheduling, customer-service systems, and CPU scheduling.

Deque: Insertion and deletion from both ends, such as task processing and undo/redo operations.

Map: Key-value relationships such as student roll number and student name.

SortedMap: Key-value pairs maintained according to sorted keys.

NavigableMap: Range searching and finding keys immediately before or after a specified key.

Iterator: Sequential traversal of collection elements.

ListIterator: Forward/backward traversal and modification of list elements.

10. RESULT

Thus, the important methods of the Java Collection Framework were studied and implemented successfully using twelve separate Java programs. The programs demonstrated insertion, deletion, searching, updating, sorting, traversal, navigation, and retrieval using the required Collection Framework interfaces. The programs were compiled and executed successfully and the required outputs were obtained.

— END OF TASK 05 —